WIZARD ENEMY

FULL ANIMATION BEHAVIOR TREE SPELL COMBAT

Create a production-ready animated wizard enemy with grounded navigation, team-based perception, a readable Behavior Tree, blackboard tuning, a non-spammable cast action, a StaffTip projectile socket, hit reactions, death, and standalone build support.

Updated workflow

- Create Task, Decorator, and Service C++ scripts directly beside their Script Class picker in the Behavior Graph.
- Rename Selector and Sequence composites with Display Name for readable combat branches.
- Attach an ordered list of gameplay scripts to the character in the Character Editor.
- Save once and use the same character, graph, scripts, animation assets, and particle assets in Play and builds.

Engine-specific setup guide - Character Editor, Clip Editor, Animation Graph, Behavior Graph, Nav Mesh Bound Volume, Particle Editor, editor Play, and exported game.

Outcome and system architecture

The finished enemy patrols while the player is unseen, focuses and chases when the player enters vision, stops at spell range, plays a full-body cast without spam, launches from the animated StaffTip socket, reacts to damage, and dies once.

System	Asset or component	Responsibility
Character	WizardEnemy.3dgcharacter	Model, materials, collider, sockets, movement, health, team, AI, scripts
Locomotion	PlayerLocomotion.3dggraph	Idle, walk, run, jump, fall, and land from movement parameters
Combat action	WizardCast.3dgclip	One-shot cast called by AI logic, not a locomotion state
Navigation	Nav Mesh Bound Volume + Nav Agent	Grounded path following, stairs, slopes, and wall avoidance
Decision logic	WizardEnemy.btgraph	Perception, focus, cast, chase, retreat, and patrol
Spell VFX	EnemyArcaneBolt.3dgparticle	Projectile appearance
Gameplay reactions	WizardLifeReactions	Hit and death Action Clips

Responsibility map

```
Nav Agent + Physics  -> grounded movement, slopes, steps, collision, Nav Mesh path
Behavior Tree        -> see player, focus, cast, chase, retreat, patrol
Animation Graph      -> reads Speed / IsGrounded / IsFalling / VerticalSpeed
WizardCastSpell Task -> stops steering, plays WizardCast, launches at StaffTip
Gameplay scripts     -> independent reusable reactions, stats, loot, audio, UI
```

Important separation: the Behavior Tree decides intent. The Animation Graph reads movement flags for locomotion. Standalone Action Clips handle casting, hit reactions, and death. Do not duplicate grounded/falling state in the blackboard.

Guide path

Stage	Result
1-2	Content layout and reusable Wizard character
3	Locomotion graph and standalone actions
4	Nav volume, grounded AI movement, perception, and teams
5-7	Blackboard and readable Behavior Tree
8-9	Inline Script Task creation and projectile implementation
10	Multiple character scripts
11-13	Assignment, testing, troubleshooting, and completion

1. Prerequisites and Content layout

Create the folders in the Content browser so every authored asset remains accessible from editor dropdowns.

```
Content/
  Assets/
    AI/WizardEnemy.btgraph
    AnimationGraphs/PlayerLocomotion.3dggraph
    AnimationActionClips/WizardCast.3dgclip
    AnimationActionClips/WizardHit.3dgclip
    AnimationActionClips/WizardDeath.3dgclip
    Characters/WizardEnemy.3dgcharacter
    Models/WizardEnemy.fbx
    Models/WizardStaff.fbx
    Particles/EnemyArcaneBolt.3dgparticle
  Scripts/
    WizardCastSpell.h
    WizardLifeReactions.h
```

Required source clips

Clip	Role	Loop	Root motion
Idle	Standing locomotion	Yes	Strip
Walk	Patrol speed	Yes	Strip
Run	Chase speed	Yes	Strip
JumpStart / JumpAir / Land	Air movement	No / Yes / No	Strip
WizardCast	Spell action	No	Strip unless deliberately authored
WizardHit	Damage reaction	No	Strip
WizardDeath	Terminal action	No	Usually strip

Preflight

- Player has Health, team 1, and a stable object name used as the Chase Target.
- Wizard stands upright and faces the engine gameplay-forward direction, local -Z.
- Ground, stairs, and walls have blocking collision.
- Nav Mesh Bound Volume covers all intended walkable surfaces.
- Projectile particle asset visibly previews before combat integration.

Skeleton compatibility: reuse the player graph only when bone names and hierarchy are compatible. If the wizard binds incorrectly, duplicate the graph and substitute wizard-compatible clips.

2. Build the Wizard Enemy character

Open **Panels > Character Editor**, choose New, and save as **Content/Assets/Characters/WizardEnemy.3dgcharacter**.

Mesh, collider, and movement

- Mesh: choose WizardEnemy.fbx and its material. Use Character Editor model transforms only for render orientation.
- Collision: enable Capsule, show its boundary, and start near radius 0.45 m and height 1.8 m.
- Movement: walk 2.2, run 4.5, turn speed 12, step height 0.35, maximum slope 50 degrees.
- Gameplay: enable Health at 80 / 80.

Staff and sockets

- Add StaffSocket on the casting-hand bone, commonly hand_r.
- Add WizardStaff.fbx as an attachment and select StaffSocket.
- Select the attachment/socket marker so the gizmo edits the socket in the preview.
- Add a second socket named exactly StaffTip and move its cyan marker to the staff tip.

Why two sockets? StaffSocket mounts the visible prop. StaffTip is an independent gameplay spawn point. Both follow the animated hand, while StaffTip remains easy to tune.

Recommended settings

Setting	Value	Reason
Collider	Capsule 0.45 x 1.8	Stable grounded movement
Walk / run	2.2 / 4.5	Readable patrol and chase
Turn speed	12	Smooth target focus
Health	80 / 80	Allows repeated combat tests
Reach radius	7.0	Ranged stopping and cast distance
Vision	16 m / 70 deg half-angle	Predictable detection cone
Team	2 (Enemy)	Hostile to player team 1

3. Full animation setup

3.1 Reuse or duplicate locomotion

- In Character Editor > Animation, choose the saved locomotion .3dggraph.
- Preview at Speed 0, walk speed, and run speed; then test grounded, falling, and vertical speed values.
- Open the Animation Graph Editor if clips are missing. Refresh clips and use unique aliases.

Parameter	Type	Runtime source	Use
Speed	Float	Horizontal movement	Idle / walk / run Blend Space
IsGrounded	Bool	Movement component	Leave jump/fall states
IsFalling	Bool	Movement component	Air-loop selection
VerticalSpeed	Float	Movement component	Rising versus falling

3.2 Author standalone Action Clips

- Create WizardCast.3dgclip in Clip Editor. Set Action Name to WizardCast.
- Use fade-in 0.10 s, fade-out 0.18 s, speed 1.0, and no mask for a full-body cast.
- Add event ReleaseSpell on the release frame for optional audio/VFX synchronization.
- Create WizardHit and WizardDeath similarly. Death should remain full-body and terminal.
- In Character Editor, refresh Standalone Action Clips, add all three, then save.

Non-spammable behavior: the Cast branch uses Cooldown, and WizardCastSpell returns Running until its action window completes. A non-layered full-body action locks character movement until completion.

Smoothness

Use Blend Space interpolation for walk/run changes, non-zero transition fades, and the authored turn speed. Physics step smoothing prevents the camera from snapping as the grounded capsule climbs stairs.

4. Navigation, grounding, perception, and teams

4.1 Nav Mesh Bound Volume

- Choose Add > AI > Nav Mesh Bound Volume and scale it around floors, corridors, and stairs.
- Build or refresh navigation and enable the AI navigation debug overlay.
- Keep stair rises below Step Height and the effective staircase slope below Max Slope.
- Leave vertical grounding to AI Movement and physics; Behavior Tree steering remains horizontal.

4.2 Character Editor AI settings

Field	Value	Notes
Nav Agent	Enabled	Required for graph-driven navigation
Speed / Max Force	4.5 / 25	Responsive but controlled chase
Reach Radius	7.0	Stop and cast distance
Repath	0.25 s	Tracks moving player
Chase Target	Player object	Explicit fallback target
Vision	16 m / 70 deg	Distance and half-angle
Team	2 / Enemy	Player should be team 1
Auto-target	On	Nearest living hostile
Movement Mode	Grounded / Falling	Gravity, floor probes, stairs
Behavior Tree	WizardEnemy.btgraph	Choose from saved-tree dropdown

4.3 Patrol and collision

- Place at least three reachable patrol waypoints inside the Nav Mesh.
- Enemy capsule must Block World Static. Coins and triggers should Overlap or Ignore Enemy.
- Test physical patrol movement before assigning complex combat behavior.

Navigation chooses a path; collision enforces physical walls. A valid Nav Mesh cannot compensate for a collision response configured to Ignore.

5. Create the Blackboard

Open **Panels > Behavior Graph**, choose New, and save as **Content/Assets/AI/WizardEnemy.btgraph**. Add these typed keys:

Key	Type	Initial	Use
aggressive	Bool	true	Designer switch for combat branch
casting	Bool	false	Live debug and cross-system state
spellDamage	Float	12.0	Projectile damage
castRelease	Float	0.42	Seconds before projectile launch
castDuration	Float	1.05	Task completion / anti-spam window
projectileSpeed	Float	11.0	Projectile travel speed

Rules

- Keys are case-sensitive. Match the spelling in nodes and C++.
- Use the Check Bool checkbox for aggressive; do not enter a float value.
- Tune blackboard values without recompiling the task.
- Watch casting in the live Blackboard view during Play.

Why these keys

The graph retains designer-owned tuning, while the custom task supplies safe defaults. The task can therefore run even when a key is temporarily missing, but the Blackboard remains the authoritative tuning surface.

6. Author a readable Wizard Behavior Tree

Create the hierarchy below. Select every Selector or Sequence and enter the quoted text in its new **Display Name** field. The runtime still uses the original composite type.

```
Selector  Display Name: Combat Priority  [ROOT]
|
+-- Sequence  Display Name: Target Dead
|   +-- Target Dead?
|   +-- Clear Focus
|   +-- Idle
|
+-- Sequence  Display Name: Low Health Retreat
|   +-- Health Below? 0.20
|   +-- Clear Focus
|   +-- Flee                [Time Limit 2.5 s]
|
+-- Sequence  Display Name: Visible Combat
|   [Check Bool aggressive = true]
|   +-- See Target?
|   +-- Focus Target
|   +-- Selector  Display Name: Cast Or Chase
|       +-- Sequence  Display Name: Cast Spell
|       |   [Target Within 7.0 m] [Cooldown 1.75 s]
|       |   +-- Set Bool casting = true
|       |   +-- Script Task WizardCastSpell
|       |   +-- Set Bool casting = false
|       +-- Chase             [Repath Service 0.25 s]
|
+-- Sequence  Display Name: Patrol Fallback
    +-- Clear Focus
    +-- Set Bool casting = false
    +-- Patrol
```

Save behavior: Save overwrites the canonical graph path. It does not append another .btgraph extension. Display Names with spaces persist in the asset.

7. Configure tasks, decorators, and services

Branch	Configuration	Reason
Target Dead	First child of root	Immediately stops combat after player death
Low Health	Health Below 0.20; Time Limit on Flee	Finite retreat and reevaluation
Visible Combat	Check Bool aggressive; See Target; Focus Target	Perception and smooth facing
Cast Spell	Target Within 7; Cooldown 1.75 on Sequence	Gates the complete cast transaction
Chase	Repath Service 0.25	Tracks a moving target
Patrol	Clear Focus before Patrol	Returns facing to locomotion

Focus and range

- Focus Target runs only after See Target? succeeds.
- Clear Focus is required in retreat, dead, and patrol branches.
- Reach Radius and Target Within should use the same casting distance.
- Chase stops while the target is in reach and resumes when the target leaves.

Decorator placement

Attach Target Within and Cooldown to the **Cast Spell Sequence**, not just to the Script Task. This gates the Set Bool writes and the task as one atomic branch.

Live debugging

- Yellow border: Running. Green: Success. Red: Failure.
- Use composite Display Names to identify the active branch without reading node IDs.
- Inspect aggressive and casting in the live Blackboard.

8. Create WizardCastSpell inside the Behavior Graph

The AI script workflow is now integrated into the graph; do not navigate to the object Inspector or manually edit GameModule registration.

Inline creation

- Select the Script Task in the Cast Spell branch.
- Beside Script Class, click **+ Create**.
- Enter class name **WizardCastSpell** and choose Create.
- The editor copies TaskTemplate.h into Content/Scripts/WizardCastSpell.h.
- The generated class is registered for editor Play and standalone builds and is assigned to the node immediately.
- Open the generated source in the built-in or preferred code editor and replace its body with the implementation on the next pages.
- Choose **Compile Scripts & Restart** once so the native registry loads the new class.

The same + Create action appears beside Script Decorator and Script Service class pickers and automatically chooses their correct templates.

Generated workflow

```
Behavior Graph Script Task
  -> + Create
  -> Content/Scripts/WizardCastSpell.h
  -> generated shared registry entry: bt WizardCastSpell
  -> Compile Scripts & Restart
  -> available in editor Play and exported project
```

8.1 WizardCastSpell - action and timing

```
#pragma once
#include <engine/ai/BtScript.h>
#include <engine/ai/BehaviorGraph.h>
#include <engine/animation/AnimatedModel.h>
#include <engine/assets/ParticleAsset.h>
#include <engine/ecs/Components.h>
#include <engine/gameplay/GameplayComponents.h>
#include <engine/graphics/ParticleSystem.h>
#include <algorithm>

class WizardCastSpell final : public engine::ai::BtScript {
public:
    void OnEnter(engine::ai::AgentContext& c) override {
        m_elapsed = 0.0f;
        m_launched = false;
        PlayCastAction(c);
    }

    engine::ai::BtStatus Tick(
        engine::ai::AgentContext& c, float dt) override {
        if (!c.registry || c.self == engine::ecs::kNull
            || c.targetEntity == engine::ecs::kNull) {
            return engine::ai::BtStatus::Failure;
        }
        const auto* target =
            c.registry->TryGet<engine::ecs::Transform>(c.targetEntity);
        if (!target) return engine::ai::BtStatus::Failure;

        c.steer = glm::vec3(0.0f);
        c.agent.velocity = glm::vec3(0.0f);
        glm::vec3 face = target->position - c.agent.position;
        face.y = 0.0f;
        if (glm::dot(face, face) > 1.0e-6f)
            c.facing = glm::normalize(face);

        m_elapsed += std::max(dt, 0.0f);
        const float release =
            c.blackboard.GetFloat("castRelease", 0.42f);
        const float duration =
            c.blackboard.GetFloat("castDuration", 1.05f);
        if (!m_launched && m_elapsed >= release)
            m_launched = LaunchProjectile(c, target->position);
        return m_elapsed >= duration
            ? engine::ai::BtStatus::Success
            : engine::ai::BtStatus::Running;
    }

    void OnExit(engine::ai::AgentContext& c) override {
        c.blackboard.SetBool("casting", false);
    }
```

Returning Running holds the branch and prevents immediate replay. Steering is zero throughout the full-body cast, while Focus Target keeps the wizard facing the player.

8.2 WizardCastSpell - Action Clip and StaffTip

```
private:
    static void PlayCastAction(engine::ai::AgentContext& c) {
        auto* animated = c.registry->TryGet<engine::AnimatedModel>(c.self);
        if (!animated || !animated->model) return;
        const auto& clips = animated->model->Animations();
        for (std::size_t i = 0; i < clips.size(); ++i) {
            if (clips[i].name == "WizardCast") {
                animated->PlayAction(
                    static_cast<int>(i), {}, {}, 0.10f, 0.18f, 1.0f);
                return;
            }
        }
    }

    static glm::vec3 ResolveStaffTip(
        engine::ai::AgentContext& c,
        const engine::ecs::Transform& owner) {
        const auto* animated =
            c.registry->TryGet<engine::AnimatedModel>(c.self);
        glm::mat4 socketWorld(1.0f);
        if (animated && animated->SocketWorldTransform(
                owner, "StaffTip", &socketWorld)) {
            return glm::vec3(socketWorld[3]);
        }
        const glm::vec3 forward =
            owner.rotation * glm::vec3(0.0f, 0.0f, -1.0f);
        return owner.position + glm::vec3(0.0f, 1.1f, 0.0f)
             + forward * 0.7f;
    }
```

Alias and forward convention

- WizardCast is the Action Name / alias, not necessarily the FBX take name.
- The fallback uses local -Z, the engine character gameplay-forward direction.
- Normal firing aims from StaffTip directly toward the player's current position.
- If the projectile starts at the body, save the Character asset, Apply to Selected, and save the scene.

8.3 WizardCastSpell - projectile and particle asset

```
    static bool LaunchProjectile(
        engine::ai::AgentContext& c,
        const glm::vec3& targetPosition) {
        auto* owner =
            c.registry->TryGet<engine::ecs::Transform>(c.self);
        if (!owner) return false;
        const glm::vec3 start = ResolveStaffTip(c, *owner);
        glm::vec3 direction =
            targetPosition + glm::vec3(0.0f, 0.8f, 0.0f) - start;
        direction = glm::dot(direction, direction) > 1.0e-6f
            ? glm::normalize(direction)
            : owner->rotation * glm::vec3(0.0f, 0.0f, -1.0f);

        const engine::ecs::Entity projectile = c.registry->Create();
        engine::ecs::Transform transform;
        transform.position = start;
        c.registry->Add<engine::ecs::Transform>(projectile, transform);

        engine::Projectile spell;
        spell.dir = direction;
        spell.speed =
            c.blackboard.GetFloat("projectileSpeed", 11.0f);
        spell.range = 20.0f;
        spell.damage = c.blackboard.GetFloat("spellDamage", 12.0f);
        spell.radius = 1.0f;
        spell.owner = c.self;
        c.registry->Add<engine::Projectile>(projectile, spell);

        engine::ParticleSystemComponent particles;
        std::string error;
        if (engine::LoadParticleAsset(
            "Content/Assets/Particles/EnemyArcaneBolt.3dgparticle",
            &particles, &error)) {
            particles.enabled = true;
            particles.autoplay = true;
            particles.loop = true;
            c.registry->Add<engine::ParticleSystemComponent>(
                projectile, std::move(particles));
        }
        return true;
    }

    float m_elapsed = 0.0f;
    bool m_launched = false;
};
```

The Projectile component supplies movement and damage even if the particle path fails. Test damage with the particle temporarily removed to separate gameplay problems from rendering problems.

9. Add multiple gameplay scripts to the Wizard

Behavior Tree scripts belong to graph nodes. Gameplay scripts belong to the character entity. The Character Editor now stores an ordered list, so reactions, stats, loot, audio, or other reusable systems can coexist.

Create WizardLifeReactions

Create the gameplay script in Content/Scripts with the normal script creator, then compile it.

```
class WizardLifeReactions final : public engine::Script {
public:
    void OnCreate() override {
        const auto* health = TryGet<engine::Health>();
        m_previousHp = health ? health->hp : 0.0f;
    }
    void OnUpdate(float) override {
        const auto* health = TryGet<engine::Health>();
        if (!health) return;
        if (!health->alive && !m_deathPlayed) {
            PlayActionClip("WizardDeath");
            m_deathPlayed = true;
        } else if (health->alive && health->hp < m_previousHp
                  && !IsAnimationActionPlaying()) {
            PlayActionClip("WizardHit");
        }
        m_previousHp = health->hp;
    }
private:
    float m_previousHp = 0.0f;
    bool m_deathPlayed = false;
};
```

Character Editor > Script

- Click + Add Script and choose WizardLifeReactions from the searchable dropdown.
- Add any other gameplay scripts needed by this character, for example WizardLootDrop or WizardAudio.
- Use Up to define execution order. Use each checkbox to enable or disable independently.
- Click Apply Scripts to Selected, save the Character asset, then save the scene.

Every enabled entry receives its own OnCreate, OnUpdate, OnFixedUpdate, timers, fields, exception isolation, and OnDestroy lifecycle in editor Play and standalone builds.

10. Place, assign, compile, and test

Placement

- Open WizardEnemy.3dgcharacter and choose Add to Scene, or Apply to Selected.
- Confirm model, capsule, Health, grounded AI Movement, sockets, Action Clips, team, and script list.
- In the AI section choose WizardEnemy.btgraph from the saved Behavior Tree dropdown.
- Choose the player Chase Target or enable hostile auto-targeting.
- Compile native scripts, restart when requested, and save the scene before Play.

Expected sequence

Test	Expected result
Player outside vision	Wizard patrols reachable waypoints
Player enters vision	Visible Combat runs; wizard focuses smoothly
Outside cast range	Chase runs and Repath follows
Inside cast range	Wizard stops, casts once, launches at StaffTip
Cooldown active	Cast branch is gated; no animation spam
Player moves away	Chase resumes
Wizard below 20 percent HP	Low Health Retreat clears focus and flees
Wizard takes damage	WizardHit plays if no higher action is active
Wizard reaches zero HP	WizardDeath plays once
Player dies	Target Dead clears focus and idles

Isolation tests

- Set aggressive=false to verify Patrol Fallback.
- Set castRelease=0.05 to verify launch timing quickly.
- Disable only WizardLifeReactions to separate reaction logic from AI combat.
- Remove VFX temporarily; the invisible Projectile should still damage the player.

11. Debugging and troubleshooting

Symptom	Likely cause	Fix
Bind pose	Graph clips missing or incompatible	Refresh clips; verify skeleton; use wizard graph
Locomotion snaps	Low Blend Space / transition smoothing	Increase interpolation, fades, turn smoothing
Sinks or flies	Wrong movement mode or capsule	Grounded/Falling; match capsule and floor probe
Crosses walls	Collision ignores World Static	Use Enemy preset and blocking wall colliders
Cannot see player	Target, team, cone, or sight blocked	Verify team 1 vs 2 and inspect AI debug
Turns late	Focus missing or branch order wrong	Place Focus Target after See Target?
Never casts	Range mismatch, aggressive false, class unloaded	Match 7 m; inspect Blackboard; compile scripts
Casts every frame	Cooldown on wrong node or task succeeds	Cooldown Cast Spell Sequence; return Running
Task missing	Created but not compiled/restarted	Compile Scripts & Restart; reopen graph
Projectile at body	StaffTip absent on placed object	Save character, Apply to Selected, save scene
Projectile backward	Using +Z fallback	Use local -Z or aim socket-to-target
Only one script runs	Character list not applied to scene	Apply Scripts to Selected and save scene
Hit interrupts cast	Reaction lacks action guard	Keep IsAnimationActionPlaying check
Death loops	No terminal guard	Keep m_deathPlayed

Debug views

- Behavior Graph: named composite border state and live Blackboard.
- AI navigation: walkable area, path, patrol points, and vision direction.
- Physics: capsule and blocking walls; use the Play-mode debug toggle.
- Character preview: StaffSocket and StaffTip markers following animation.
- Console: missing registry class, particle path, Action Clip alias, or asset load errors.

12. Completion checklist

Character and animation

- WizardEnemy.3dgcharacter saved and applied to the placed object.
- Locomotion graph verified with compatible clips and smooth Blend Space transitions.
- WizardCast, WizardHit, and WizardDeath appear under Standalone Action Clips.
- Staff follows StaffSocket and StaffTip marker sits at the tip.
- Ordered gameplay-script list is saved and applied.

Navigation and behavior

- Nav Mesh Bound Volume covers intended floors and stairs.
- Wizard has grounded movement, blocking capsule, team 2, and reachable patrol points.
- WizardEnemy.btgraph is chosen from the character's Behavior Tree dropdown.
- Selector and Sequence Display Names make every branch readable.
- Tasks, decorators, Repath service, and Blackboard keys match the guide.

Combat and build

- WizardCastSpell was created with the graph's + Create control and compiled.
- Cast cannot spam; movement remains stopped for the full-body action.
- Projectile launches from animated StaffTip and travels toward the player.
- Player Health decreases; hit and death actions follow correct priority.
- The saved scene validates, exports, and runs identically in the standalone player.

Finished result
The Wizard Enemy now combines animation-driven locomotion, grounded navigation, team-aware perception, a readable named Behavior Tree, inline AI script creation, blackboard tuning, non-spammable socket-based spell casting, projectile damage, and multiple independent gameplay scripts.